Analise de Eficacia de Testes com Teste de Mutacao

Relatorio do Trabalho Pratico

Disciplina: Testes de Software

Nome completo: Caio Victor Kodato Teixeira

Matricula: 787216

Repositorio: https://github.com/CaioVKodato/operacoes-mutante

Ferramenta utilizada: StrykerJS + Jest

1. Analise Inicial

O projeto base (biblioteca de 50 operacoes matematicas) possui uma suite de testes deliberadamente fraca: os testes exercitam o codigo, mas verificam pouco o comportamento. Isso gera alta cobertura com baixa eficacia real.

1.1 Cobertura de codigo inicial

Statements:	85.41%
Branches:	58.82%
Functions:	100%
Lines:	98.64%

A cobertura de linhas ja era quase total (~98,64%), o que poderia sugerir, erroneamente, que a suite era boa. A cobertura de branches, no entanto, ja indicava lacunas (58,82%).

1.2 Pontuacao de mutacao inicial

Mutation Score:	73.71%
Mutantes gerados:	213
Killed:	154
Timeout:	3
Survived:	44
NoCoverage:	12

Ha uma discrepancia clara: cobertura de linhas alta (~98%) versus mutation score moderado (73,71%). Isso confirma a limitacao da cobertura: ela mede se o codigo foi executado, nao se as assercoes detectariam um bug. Mutantes sobreviventes mostram testes que 'passam pelo codigo' sem validar o resultado de forma rigorosa.

2. Analise de Mutantes Criticos

A seguir, tres mutantes sobreviventes da primeira execucao do Stryker, escolhidos por ilustrarem fraquezas tipicas de suites fracas.

2.1 Mutante A - Comparacao fraca (isMaiorQue)

Trecho original:

```
function isMaiorQue(a, b) { return a > b; }
```

Mutacao aplicada pelo Stryker (EqualityOperator):

```
function isMaiorQue(a, b) { return a >= b; }
```

Por que sobreviveu: o teste original apenas fazia expect(isMaiorQue(10, 5)).toBe(true). Com a >= b, o resultado para 10 e 5 continua true. Nao havia caso em que a === b, no qual > retorna false e >= retorna true. Sem esse caso de fronteira, o mutante nao e morto.

Evidencia no relatorio Stryker (status Survived, linha 104 de operacoes.js).

2.2 Mutante B - Assercao incompleta em booleano (isPar)

Trecho original:

```
function isPar(n) { return n % 2 === 0; }
```

Mutacao aplicada (ConditionalExpression):

```
function isPar(n) { return true; }
```

Por que sobreviveu: o teste so verificava o caminho verdadeiro (expect(isPar(100)).toBe(true)). Como o mutante tambem retorna true nesse caso, o teste continua passando. Faltava um caso negativo (numero impar) para forcar a falha.

2.3 Mutante C - Conversao termometrica com entrada degenerada

Trecho original:

```
function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; }
```

Mutacoes que sobreviveram (ArithmeticOperator):

```
(celsius * 9 * 5) + 32
(celsius / 9 / 5) + 32
```

Por que sobreviveram: o teste usava apenas celsius = 0. Qualquer multiplicacao/divisao por constante com zero permanece zero; o resultado continua 32. A entrada 0 mascara erros aritmeticos. Um valor nao nulo (ex.: 100 -> 212) mata esses mutantes imediatamente.

3. Solucao Implementada

A suite foi reforçada com assercoes mais especificas: casos de fronteira, caminhos booleanos verdadeiros e falsos, mensagens de erro exatas e entradas que tornam mutacoes aritmeticas observaveis.

3.1 Exemplo para o Mutante A (isMaiorQue)

```
expect(isMaiorQue(10, 5)).toBe(true);
expect(isMaiorQue(5, 10)).toBe(false);
expect(isMaiorQue(5, 5)).toBe(false); // mata a > b  ->  a >= b
```

O caso a === b distingue > de >=. Qualquer mutacao que afrouxa o operador falha neste teste.

3.2 Exemplo para o Mutante B (isPar / isImpar)

```
expect(isPar(100)).toBe(true);
expect(isPar(7)).toBe(false);
expect(isImpar(7)).toBe(true);
expect(isImpar(4)).toBe(false);
```

Cobrir true e false impede mutantes que forcam retorno constante (return true / return false).

3.3 Exemplo para o Mutante C (Celsius)

```
expect(celsiusParaFahrenheit(0)).toBe(32);
expect(celsiusParaFahrenheit(100)).toBe(212);
expect(celsiusParaFahrenheit(37)).toBeCloseTo(98.6);
```

Valores nao nulos tornam observavel a troca de * por / (e vice-versa) na formula.

3.4 Outras melhorias relevantes

- Casos de array vazio (media, produto, max/min, mediana).
- isPrimo: 1, 0, negativos e compostos (9, 4, 15), alem de primos.
- clamp: abaixo do minimo, acima do maximo e valores internos.
- mediana: tamanho impar, par e array desordenado.
- Alguns mutantes restantes eram equivalentes (ex.: < vs <= em clamp quando o retorno no limite e o mesmo). Foram documentados com comentarios Stryker disable, pratica recomendada para mutantes equivalentes.

4. Resultados Finais

4.1 Cobertura apos melhoria

Statements:	100%
Branches:	100%
Functions:	100%
Lines:	100%

4.2 Mutation score final

Mutation Score:	100.00%
Killed:	199
Timeout:	3
Survived:	0
NoCoverage:	0

Evolucao: 73,71% -> 100,00% (meta da atividade: > 98%). A melhoria veio de assercoes mais fortes, nao apenas de 'passar pelo codigo'. Relatorios HTML do Stryker estao em reports/mutation/ (mutation-inicial.html e mutation.html).

5. Conclusao

Este trabalho evidencia que cobertura de codigo e eficacia de testes sao metricas diferentes. Uma suite pode atingir quase 100% de cobertura de linhas e ainda assim deixar passar dezenas de mutantes, porque nao verifica o comportamento com rigor.

O teste de mutacao com StrykerJS expoe exatamente essas falhas: ele introduz bugs sinteticos e pergunta se a suite os detectaria. Ao escrever testes focados em matar mutantes (fronteiras, caminhos negativos, mensagens e valores nao degenerados), a qualidade da suite aumenta de forma mensuravel pelo Mutation Score.

Como engenheiro de qualidade, a licao pratica e: use cobertura para achar codigo nao exercitado, e use mutacao para avaliar se as assercoes realmente protegem o comportamento.